

CHAPTER

Working with Databases in Python

SQLite · PostgreSQL · MongoDB · SQLAlchemy ORM · CRUD Operations · Row Factories · Dataclasses

Learning Objectives

By the end of this chapter, you will be able to:

- Explain the distinction between SQL and NoSQL databases and when to use each.
- Create, connect to, and perform full CRUD operations on SQLite databases using Python's sqlite3 module.
- Write parameterized SQL queries to prevent SQL injection vulnerabilities.
- Apply row factories and Python dataclasses to transform raw query results into structured objects.
- Connect to and query a PostgreSQL server using psycopg2.
- Perform CRUD operations on MongoDB-style document databases using Mongita and pymongo.
- Use advanced MongoDB query operators, embedded documents, arrays, and update operators.
- Define ORM models, create engines, manage sessions, and query databases using SQLAlchemy.
- Model one-to-many relationships using SQLAlchemy's relationship() and foreign keys.

SECTION 1 — THEORETICAL FOUNDATIONS

1.1 Introduction to Databases in Python

At the core of every real-world application — whether a financial portfolio tracker, an e-commerce platform, or a machine learning data pipeline — lies a **database**: a structured system for storing, organizing, retrieving, and updating large volumes of data efficiently and reliably. Without databases, data would exist only in memory, evaporating when a program terminates.

Python's rich ecosystem of database libraries makes it one of the most powerful languages for data-intensive work. Depending on the nature and scale of your data, Python can connect to **local file-based databases** (like SQLite), **enterprise relational servers** (like PostgreSQL and MySQL), and **distributed document stores** (like MongoDB). In the Data Science and GenAI context, databases serve as the persistent backbone for training data, model artifacts, user interactions, and application state.

1.1.1 SQL vs. NoSQL — A Fundamental Distinction

Understanding the architectural difference between SQL and NoSQL databases is essential for choosing the right tool for a given problem.

Dimension	SQL (Relational)	NoSQL (Non-Relational)
Data Model	Tables with rows and columns — a rigid, predefined schema	Documents, key-value pairs, graphs, or columns — flexible structures
Schema	Fixed schema; altering it requires migrations	Schema-optional or schema-flexible; documents in one collection can differ in structure
Query Language	Structured Query Language (SQL) — standardised across databases	Database-specific APIs (MongoDB query language, Redis commands, Cypher for Neo4j)
Relationships	First-class support via JOIN operations and foreign keys	Typically handled via embedding or application-level logic
ACID Compliance	Strong ACID guarantees (Atomicity, Consistency, Isolation, Durability)	Varies by system; many NoSQL databases prioritise availability and partition tolerance (CAP theorem)
Best Suited For	Financial systems, ERP,	Content management, real-

	inventory, structured analytics	time apps, unstructured/semi- structured data, horizontal scaling
Python Examples	sqlite3, psycopg2 (PostgreSQL), PyMySQL (MySQL), SQLAlchemy ORM	pymongo (MongoDB), redis-py (Redis), cassandra- driver

Key Insight: Complementary, Not Competing

SQL and NoSQL databases are not adversaries — they are complementary tools. A modern Data Science application might use PostgreSQL to store structured user and transaction data (benefiting from ACID guarantees and JOINS), while simultaneously using MongoDB to store unstructured model outputs and logs (benefiting from schema flexibility). Choosing the right database for each use case is an architectural decision, not a tribal allegiance.

1.2 SQLite — The Embedded Relational Database

SQLite is a **self-contained, serverless, zero-configuration relational database engine** that stores an entire database in a single cross-platform file. Unlike client-server databases (PostgreSQL, MySQL), SQLite requires no separate database process — it runs directly within the host application process. This makes SQLite the most widely deployed database in the world, embedded in every Android and iOS device, every modern web browser, and countless desktop applications.

Python includes SQLite support in the standard library via the `sqlite3` module — no installation is required. This makes SQLite the perfect choice for rapid prototyping, local development, unit testing, and standalone tools where setting up a server would be unnecessary overhead.

1.2.1 The Connection-Cursor Architecture

Interacting with any database in Python follows a consistent architectural pattern: the **Connection-Cursor model**. The `connection` object represents an open channel to the database file and manages transactions. The `cursor` object is obtained from the connection and serves as the active agent for executing SQL statements and fetching results. A single connection can produce multiple cursors, allowing parallel query execution.

1.2.2 SQLite Data Types

SQLite uses a dynamic type system with five storage classes that map to Python types as follows:

SQLite Type	Python Equivalent & Description
TEXT	str — Variable-length Unicode text. Used for names, identifiers, and any string data.
INTEGER	int — Signed integer, stored in 1–8 bytes depending on magnitude.
REAL	float — IEEE 754 floating-point number stored in 8 bytes.
BLOB	bytes — Binary data stored exactly as provided. Used for files, images.
NULL	None — Represents a missing or unknown value.
TIMESTAMP	datetime.datetime — Python's sqlite3 adapter auto-converts datetime objects.
BOOLEAN	bool — Stored as 0 (False) or 1 (True). Python's adapter handles conversion.

1.2.3 SQL Injection and Parameterized Queries

SQL injection is one of the most dangerous and prevalent security vulnerabilities in database-driven applications. It occurs when user-supplied input is concatenated directly into a SQL string, allowing an attacker to modify the query's logic. For example, a username input of `' OR '1'='1` could bypass authentication entirely.

The solution is **parameterized queries** (also called prepared statements). Instead of formatting values into the SQL string yourself, you pass them as a separate tuple. The database driver handles all escaping and quoting internally — it is **structurally impossible** for user input to alter the query's SQL logic when parameterized queries are used. In Python's `sqlite3`, the placeholder character is `?`.

Security Rule: Never Use String Formatting for SQL

Never build SQL queries using Python string formatting (`f"-strings"` , `%` , `.format()`). Always use parameterized queries with the `?` placeholder. This applies universally — to SQLite, PostgreSQL (which uses `%s` as placeholder), and any other database. This rule is non-negotiable in production code.

1.2.4 Row Factories

By default, `sqlite3` returns query results as plain **tuples**. Accessing data by position (`row[0]` , `row[2]`) is fragile — reordering columns in your SELECT query silently breaks all downstream code. Row factories solve this by transforming each returned row into a richer Python object before it is handed to your application.

The most commonly used row factory is `sqlite3.Row`, which provides named column access: `row['coin_id']` instead of `row[0]`. For more structured applications, a custom row factory can construct Python dataclass or named tuple instances directly from query results.

1.2.5 Python Dataclasses

Python's `@dataclass` decorator (introduced in Python 3.7, PEP 557) automatically generates boilerplate methods — `__init__()`, `__repr__()`, `__eq__()` — for a class based on its annotated fields. This eliminates repetitive constructor code and produces clean, readable data container classes. When used as row factory targets, dataclasses transform raw database rows into typed, inspectable Python objects, bridging the relational and object worlds elegantly.

1.3 PostgreSQL — The Enterprise Relational Database

PostgreSQL (often called **Postgres**) is a powerful, open-source, object-relational database system with over 35 years of active development. Unlike SQLite, PostgreSQL operates as a **standalone network server process** that client applications connect to over TCP/IP. It supports full ACID compliance, advanced indexing (B-tree, Hash, GiST, GIN), JSONB columns, stored procedures, triggers, and replication — making it the database of choice for production web applications, financial systems, and data warehouses.

PostgreSQL requires **authentication credentials** (host, database name, username, password) for every connection. This is a fundamental security difference from SQLite's file-based model. In Data Science contexts, PostgreSQL is commonly used to store cleaned, processed datasets and feature stores that are shared across teams and services.

1.3.1 psycopg2 — Python's PostgreSQL Adapter

psycopg2 is the most widely used PostgreSQL adapter for Python. It implements the DB-API 2.0 specification (PEP 249), meaning its interface is structurally identical to `sqlite3` — the same `connection.cursor()` → `cursor.execute()` → `cursor.fetchall()` pattern applies. This is by design: PEP 249 standardises the interface so switching database backends requires minimal code changes. Install via `pip install psycopg2-binary` (the binary package bundles compiled C extensions).

1.4 NoSQL Databases — MongoDB, Mongita, and Document Storage

NoSQL databases represent a paradigm shift from the rigid, schema-bound relational model. Rather than fitting data into predefined tables, NoSQL systems store data in formats native to how applications actually use it — documents, key-value pairs, graphs, or wide columns. This flexibility makes NoSQL databases especially well-suited to scenarios involving **rapidly evolving schemas**, **hierarchically structured data**, and **horizontal scalability** across distributed clusters.

NoSQL Type	Representative System	Ideal Use Case
Document Store	MongoDB, CouchDB	Content management, user profiles, product catalogues, ML feature stores
Key-Value Store	Redis, DynamoDB	Caching, session management, real-time leaderboards, pub/sub messaging
Column-Oriented	Cassandra, HBase	Time-series data, IoT sensor streams, analytical workloads at massive scale
Graph Database	Neo4j, Amazon Neptune	Social networks, recommendation engines, fraud detection, knowledge graphs

1.4.1 MongoDB's Document Model

MongoDB organises data in a three-level hierarchy: **Database** → **Collection** → **Document**. A **document** is a JSON-like structure (stored internally as BSON — Binary JSON) that can contain strings, numbers, arrays, booleans, null values, and **nested (embedded) documents**. Documents within the same collection are **not required to have the same structure** — one document may have 3 fields, another 15, and both coexist naturally.

The direct analogies between SQL and MongoDB terminology are:

SQL Concept	MongoDB Equivalent
Database	Database
Table	Collection
Row / Record	Document
Column	Field
Primary Key	_id (ObjectId, auto-generated)
JOIN	Embedded document or \$lookup aggregation
WHERE clause	Query filter (JSON-like object)
INDEX	Index (also supported in MongoDB)

1.4.2 Mongita — The Embedded MongoDB-Compatible Store

Mongita is an embedded, disk-based NoSQL database that implements a subset of the MongoDB API. Its relationship to MongoDB mirrors SQLite's relationship to PostgreSQL — it is a lightweight, dependency-free, file-stored version of the larger system, ideal for development, testing, and small standalone applications. Because Mongita uses the same API surface as pymongo (MongoDB's official Python driver), code written against Mongita can be migrated to a full MongoDB deployment with minimal changes.

1.4.3 MongoDB Query Operators

MongoDB's query language uses a JSON-based syntax with special operator keys prefixed by \$. The most important operators are:

Operator	Type & Description
	Comparison — greater than, >=, less than, <=. Used in field queries.
	Equality/Inequality — match or exclude a specific value.
	Membership — field value is in (or not in) a specified array.
	Logical AND — all conditions in the array must be true.
	Logical OR — at least one condition in the array must be true.
	Field existence — matches documents where the field exists (or doesn't).
	Update — increments a numeric field by a specified amount.
	Update — sets the value of a field (creates it if it doesn't exist).
	Array update — appends an element to an array field.
	Array update — removes all occurrences of a value from an array field.

1.5 SQLAlchemy ORM — Object Relational Mapping

SQLAlchemy is Python's most powerful and widely used database toolkit. It operates at two levels: a **Core layer** providing SQL expression constructs, and an **ORM layer** (Object Relational Mapper) that maps Python classes to database tables. We focus on the ORM, which is the layer most relevant to application development and Data Science pipelines.

1.5.1 What is an ORM?

An **Object Relational Mapper** (ORM) is a software layer that translates between the object-oriented world of Python and the relational world of SQL databases. The core mapping is:

Database Concept	ORM (Python) Equivalent
Table	Model class (inheriting from Base)
Row	Instance of the model class
Column	Class attribute (mapped_column)
INSERT	session.add(instance)
SELECT	session.execute(select(Model))
UPDATE	Modify attributes of a loaded instance, then session.commit()
DELETE	session.delete(instance)
JOIN	relationship() with back_populates

The practical benefit is that developers work primarily with Python objects and methods, while SQLAlchemy generates the corresponding SQL automatically. This improves developer productivity, reduces SQL syntax errors, enables database portability (switch from SQLite to PostgreSQL by changing one connection string), and makes complex query logic testable as pure Python.

1.5.2 The Declarative Base and Model Definition

Modern SQLAlchemy (2.0+) uses the **Declarative Base** pattern. Every ORM model inherits from a **Base** class created by subclassing **DeclarativeBase**. Each model class declares its table name via `__tablename__` and its columns via `mapped_column()` with explicit type annotations. SQLAlchemy reads these annotations to generate the correct SQL DDL (CREATE TABLE statements).

1.5.3 Engine and Session

The **engine** is the connection factory — it manages the pool of database connections and is created once per application using `create_engine()` with a database URL string (e.g., `"sqlite:///data.db"` or `"postgresql://user:pass@host/dbname"`). The **session** is the primary interface for ORM operations — it tracks all loaded objects, accumulates changes, and flushes them to the database on `commit()`. The session acts as a **unit of work**: all changes within a session are treated as a single atomic transaction.

1.5.4 Relationships and Foreign Keys

SQLAlchemy models can express **relationships** between tables using `relationship()` and `ForeignKey`. The most common pattern is **one-to-many**: one Portfolio contains many

Investments. The `back_populates` parameter on both sides of the relationship keeps the objects synchronized — adding an Investment to `portfolio.investments` automatically sets `investment.portfolio` to the parent Portfolio object. SQLAlchemy handles all JOIN SQL behind the scenes.

SECTION 2 — PRACTICAL & CODING REFERENCE

This section walks through every concept from Section 1 with complete, annotated code cells. Each cell covers the full code, the concept behind it, the rationale for the approach, and a detailed output walkthrough. All examples use a consistent financial portfolio domain — tracking cryptocurrency investments.

Part A — SQLite with sqlite3

Cell 1 — Connecting to SQLite and Creating a Cursor

```
import sqlite3
import datetime

# Connect to a database file (creates it if it doesn't exist)
database = sqlite3.connect("portfolio.db")

# Obtain a cursor — the agent for executing SQL statements
cursor = database.cursor()

print("Connection established successfully.")
print(f"SQLite version: {sqlite3.sqlite_version}")
```

Concept

The `sqlite3.connect()` call does two things: if `portfolio.db` exists on disk, it opens that file; if it does not exist, SQLite creates it as an empty database file. A special in-memory database can be created with `sqlite3.connect(":memory:")` — ideal for testing because it disappears when the connection closes. The `cursor()` method returns a `Cursor` object — the interface through which all SQL statements are issued and results are read.

Why This Approach?

Using a named file (`portfolio.db`) creates a persistent database that survives program restarts — essential for any real application. The `:memory:` variant is invaluable in unit

tests: each test starts with a clean, empty database with no disk I/O overhead. The `sqlite3.sqlite_version` attribute shows the underlying SQLite C library version, useful for diagnosing feature availability.

Expected Output & Explanation

Connection established successfully.

SQLite version: 3.45.0 (version will vary by system)

The file `portfolio.db` is created in the current working directory. No SQL has been executed yet — the database is empty.

Cell 2 — Creating a Table with DDL

```
# DDL (Data Definition Language) – defines the structure of data
create_table_query = """
CREATE TABLE IF NOT EXISTS investments (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    coin_id       TEXT      NOT NULL,
    currency      TEXT      NOT NULL DEFAULT 'usd',
    sell          INTEGER NOT NULL DEFAULT 0,
    amount        REAL      NOT NULL,
    date          TIMESTAMP
);
"""

cursor.execute(create_table_query)
database.commit()

print("Table 'investments' created (or already exists).")

# Verify: query the SQLite schema metadata
cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
tables = cursor.fetchall()
print(f"Tables in database: {tables}")
```

Concept

The **CREATE TABLE IF NOT EXISTS** guard prevents an error if the table already exists — the statement silently succeeds. Each column definition specifies: name, SQLite type, and

optional constraints. **INTEGER PRIMARY KEY AUTOINCREMENT** creates a column that auto-assigns a unique, incrementing integer ID to each new row. **NOT NULL** prevents NULL values from being inserted. **DEFAULT 'usd'** assigns a fallback value if the column is omitted in an INSERT. The **sqlite_master** table is a special SQLite system table that stores the schema of all objects (tables, indexes, views) in the database.

Why This Approach?

The **IF NOT EXISTS** clause makes this statement idempotent — safe to execute multiple times (e.g., every time the application starts). Adding an explicit **id** primary key and **NOT NULL** constraints is best practice: it ensures referential integrity and makes future JOINS and updates straightforward. The **commit()** call is required after DDL statements in **sqlite3** to persist the schema change to the file.

Expected Output & Explanation

Table 'investments' created (or already exists).

Tables in database: [('investments',)]

The **sqlite_master** query confirms the table was created. The result is a list of tuples — each tuple contains the name of one table in the database.

Cell 3 — Inserting Data with Parameterized Queries

```
import datetime

# --- SAFE: Parameterized query using ? placeholders ---
investment = (
    "bitcoin",
    "usd",
    True,          # sell flag
    1.0,          # amount in BTC
    datetime.datetime.now()
)

cursor.execute(
    "INSERT INTO investments (coin_id, currency, sell, amount, date) "
    "VALUES (?, ?, ?, ?, ?);",
    investment
)

database.commit()
```

```
# --- Insert multiple rows efficiently with executemany ---
more_investments = [
    ("ethereum", "usd", False, 5.0, datetime.datetime.now()),
    ("solana", "usd", False, 20.0, datetime.datetime.now()),
    ("cardano", "inr", True, 500.0, datetime.datetime.now()),
]

cursor.executemany(
    "INSERT INTO investments (coin_id, currency, sell, amount, date) "
    "VALUES (?, ?, ?, ?, ?);",
    more_investments
)

database.commit()

print(f"Rows inserted: {cursor.rowcount}")
print(f"Last inserted ID: {cursor.lastrowid}")
```

Concept

The `?` placeholders are the key to parameterized queries. The Python tuple `investment` is passed as the second argument to `cursor.execute()` — the driver substitutes each `?` with the corresponding value, handling all escaping automatically. `executemany()` is a performance optimization for batch inserts — it iterates over the list of tuples and executes the INSERT once per tuple, much faster than calling `execute()` in a loop. `database.commit()` is mandatory after DML (Data Manipulation Language — INSERT/UPDATE/DELETE) to persist changes; without it, changes exist only in a transaction buffer and will be lost when the connection closes.

Why This Approach?

The parameterized approach is not just best practice — it is the **only acceptable approach** for production code. Consider the alternative: `f"INSERT INTO investments VALUES ({coin_id}, ...)"`. If `coin_id` were `' ; DROP TABLE investments; --`, the table would be destroyed. Parameterized queries make such attacks structurally impossible. `executemany()` is ideal for loading batch data from CSVs or APIs into the database.

Expected Output & Explanation

Rows inserted: 3 (rowcount reflects the last executemany call)
Last inserted ID: 4 (auto-increment ID of the last inserted row)

All 4 rows are now persisted in portfolio.db. The `commit()` call flushes the transaction to disk.

Cell 4 — Querying Data — SELECT, fetchall, fetchone

```
# --- SELECT all rows ---
result = cursor.execute("SELECT * FROM investments;")
all_rows = result.fetchall()

print(f"Total records: {len(all_rows)}")
print("\nAll investments (as tuples):")
for row in all_rows:
    print(f"    {row}")

# --- SELECT with WHERE filter ---
cursor.execute(
    "SELECT coin_id, amount FROM investments WHERE currency = ?;",
    ("usd",)      # note: single-element tuple requires trailing comma
)
usd_rows = cursor.fetchall()
print(f"\nUSD investments: {usd_rows}")

# --- fetchone: retrieve a single row ---
cursor.execute(
    "SELECT * FROM investments WHERE coin_id = ?;",
    ("bitcoin",)
)
bitcoin_row = cursor.fetchone()
print(f"\nBitcoin record: {bitcoin_row}")

# --- Aggregate query ---
cursor.execute("SELECT SUM(amount), COUNT(*) FROM investments;")
total_amount, count = cursor.fetchone()
print(f"\nTotal amount: {total_amount}, Record count: {count}")
```

Concept

After `cursor.execute()` runs a SELECT, the cursor holds the result set. `fetchall()` retrieves **all remaining rows** as a list of tuples. `fetchone()` retrieves only the **next single row** — useful when you expect exactly one result or want to stream large results one at a time. The WHERE clause with a `?` placeholder filters results server-side before returning data to Python — always preferred over fetching all rows and filtering in Python. The trailing comma in `("usd",)` is required to make a single-element tuple — without it, `("usd")` is just a parenthesized string, not a tuple, causing a `TypeError`.

Why This Approach?

Server-side filtering (WHERE clause) is dramatically more efficient than Python-side filtering, especially for large tables. Fetching only the columns you need (`SELECT coin_id, amount` vs `SELECT *`) reduces data transfer. `fetchone()` is preferred when you know at most one row matches — it signals intent clearly and avoids fetching unnecessary data. Aggregate functions (SUM, COUNT, AVG, MAX, MIN) are SQL's native tools for summarising data — always use them instead of loading all rows and computing in Python.

Expected Output & Explanation

Total records: 4

All investments (as tuples): each row is a tuple like (1, 'bitcoin', 'usd', 1, 1.0, '2024-...')

USD investments: [('bitcoin', 1.0), ('ethereum', 5.0), ('solana', 20.0)]

Bitcoin record: (1, 'bitcoin', 'usd', 1, 1.0, '2024-...')

Total amount: 526.0, Record count: 4 (1.0 + 5.0 + 20.0 + 500.0)

Cell 5 — Row Factories — Named Column Access

```
# Without row factory: access by position (fragile)
cursor.execute("SELECT * FROM investments LIMIT 1;")
row = cursor.fetchone()
print(f"Positional: coin_id={row[1]}, amount={row[4]}")

# --- Use sqlite3.Row factory for named access ---
database.row_factory = sqlite3.Row

# Must re-create cursor after changing the row_factory
cursor = database.cursor()
cursor.execute("SELECT * FROM investments;")
rows = cursor.fetchall()

print("\nNamed column access (sqlite3.Row):")
```

```

for row in rows:
    print(f"  coin: {row['coin_id']:12s} | amount: {row['amount']:8.2f} |
currency: {row['currency']}")

# --- Custom row factory: returns a Python dict ---
def dict_row_factory(cursor, row):
    """Converts a SQLite row into a Python dictionary."""
    fields = [description[0] for description in cursor.description]
    return dict(zip(fields, row))

database.row_factory = dict_row_factory
cursor = database.cursor()
cursor.execute("SELECT * FROM investments LIMIT 2;")
dict_rows = cursor.fetchall()

print("\nDict rows:")
for d in dict_rows:
    print(f"  {d}")

```

Concept

The `row_factory` attribute on the **connection** (not the cursor) is a callable that `sqlite3` calls with two arguments — the cursor and the raw tuple row — for every row it returns. Setting it to `sqlite3.Row` makes every row behave like both a tuple (positional access works) and a dictionary (named access via column name works). The custom `dict_row_factory` uses `cursor.description` — a tuple of 7-item sequences where `description[0]` is the column name — to build a **mapping of column names to values**. The cursor must be re-created after changing the `row_factory` because existing cursors retain the old factory.

Why This Approach?

Named column access is fundamentally more maintainable than positional access. If the `SELECT` query is ever reordered (`SELECT amount, coin_id ...` instead of `SELECT coin_id, amount ...`), positional code silently breaks; named code works correctly. The dictionary row factory is especially useful when feeding query results directly into pandas (`pd.DataFrame(cursor.fetchall())`) or JSON serialization, as both expect dictionary-like objects.

Expected Output & Explanation

Positional: `coin_id=bitcoin, amount=1.0` (works but fragile)

Named column access output (one line per investment):

coin: bitcoin | amount: 1.00 | currency: usd

coin: ethereum | amount: 5.00 | currency: usd ...

Dict rows: [{'id': 1, 'coin_id': 'bitcoin', 'currency': 'usd', 'sell': 1, 'amount': 1.0, 'date': ...}, ...]

Cell 6 — Dataclasses as Row Factories

```
from dataclasses import dataclass
import datetime

@dataclass
class Investment:
    id: int
    coin_id: str
    currency: str
    sell: bool
    amount: float
    date: datetime.datetime

    @property
    def position_value(self):
        """Computed property – not a DB column."""
        return f"{self.amount} {self.coin_id.upper()} "
        ({self.currency.upper()})"

def investment_row_factory(cursor, row):
    """Converts a DB row tuple into an Investment dataclass instance."""
    fields = [col[0] for col in cursor.description]
    row_dict = dict(zip(fields, row))
    return Investment(**row_dict)

# Apply the custom row factory
database.row_factory = investment_row_factory
cursor = database.cursor()

cursor.execute("SELECT * FROM investments;")
investments = cursor.fetchall()
```

```

print("Investments as typed dataclass objects:")
for inv in investments:
    print(f"    {inv.coin_id:12s} | {inv.position_value} | sell={inv.sell}")

# Access as a proper object
first = investments[0]
print(f"\nType: {type(first)}")
print(f"Is dataclass: {hasattr(first, '__dataclass_fields__')}")

```

Concept

The `@dataclass` decorator reads the class's type-annotated attributes and automatically generates `__init__(self, id, coin_id, currency, sell, amount, date)`, `__repr__()` (clean string representation), and `__eq__()` (equality comparison). The `investment_row_factory` function bridges the database and the dataclass: it uses `cursor.description` to get column names, zips them with the row values into a dictionary, and unpacks that dictionary as keyword arguments to `Investment(**row_dict)`. The `@property` decorator adds a computed attribute that doesn't exist in the database but is derived from stored attributes — demonstrating how dataclasses extend beyond simple data containers.

Why This Approach?

This pattern is the foundation of modern Python database patterns. In production applications, database rows are immediately converted to typed objects at the data access layer — the rest of the application never touches raw tuples. This enables IDE autocompletion (`inv.coin_id` vs `row[1]`), static type checking, and computed properties. The pattern directly mirrors how ORMs like SQLAlchemy work internally, making this an excellent stepping stone towards understanding ORM architecture.

Expected Output & Explanation

```

Investments as typed dataclass objects:
    bitcoin    | 1.0 BITCOIN (USD) | sell=True
    ethereum   | 5.0 ETHEREUM (USD) | sell=False ...
Type: <class '__main__.Investment'> (a proper Python object, not a tuple)
Is dataclass: True (confirms __dataclass_fields__ meta-attribute exists)

```

Part B — PostgreSQL with psycopg2

Cell 7 — Connecting to PostgreSQL and Executing Queries

```
import psycopg2

# --- Establish connection to PostgreSQL server ---
connection = psycopg2.connect(
    host      = "localhost",
    database = "portfolio_db",
    user      = "postgres",
    password = "your_password_here"
)

# --- Create a cursor and execute a query ---
cursor = connection.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS investments (
        id          SERIAL PRIMARY KEY,
        coin_id     VARCHAR(32) NOT NULL,
        currency    CHAR(3)     NOT NULL DEFAULT 'usd',
        amount      NUMERIC(18,8) NOT NULL
    );
""")
connection.commit()

# --- Insert and query ---
cursor.execute(
    "INSERT INTO investments (coin_id, currency, amount) VALUES (%s, %s, %s)",
    ("bitcoin", "usd", 1.5)
)
connection.commit()

cursor.execute("SELECT * FROM investments;")
data = cursor.fetchall()
print("PostgreSQL records:")
for row in data:
    print(f"    {row}")

# --- Always close resources ---
```

```

cursor.close()
connection.close()
print("\nConnection closed.")

```

Concept

The `psycopg2` interface deliberately mirrors `sqlite3` — both implement PEP 249. Key differences: PostgreSQL uses `SERIAL` for auto-increment integers (SQLite uses `INTEGER PRIMARY KEY AUTOINCREMENT`). PostgreSQL's parameterized query placeholder is `%s` (not `?`). `VARCHAR(32)` and `CHAR(3)` are PostgreSQL's typed string columns. `NUMERIC(18,8)` stores decimal values with up to 18 digits total and 8 after the decimal point — important for financial calculations where floating-point precision errors are unacceptable. Resources (cursor, connection) should always be explicitly closed or managed with a context manager.

Why This Approach?

The identical structure between `sqlite3` and `psycopg2` is intentional — PEP 249 standardised the DB-API so that Python database code is portable. The main change needed to migrate from SQLite to PostgreSQL is: change the `connect()` call, change `?` to `%s` as placeholder, and adjust DDL for PostgreSQL-specific types. Using `cursor.close()` and `connection.close()` or, better, wrapping in a `with` context manager ensures database connections are returned to the pool and not leaked.

Expected Output & Explanation

PostgreSQL records:

(1, 'bitcoin', 'usd', Decimal('1.50000000')) (NUMERIC type returns Python Decimal)

Connection closed.

Note: This cell requires a running PostgreSQL server. If not available, the concepts are identical to SQLite — only the connection parameters and placeholder syntax differ.

Part C — NoSQL with Mongita and pymongo

Cell 8 — Mongita — Embedded Document Database (CRUD)

```

from mongita import MongitaClientDisk

# Connect to Mongita (stores data in local files)
client = MongitaClientDisk()
db = client.portfolio          # access/create database
investments = db.investments  # access/create collection

```

```
# --- CREATE: insert_one ---
result = investments.insert_one({
    "coin_id" : "bitcoin",
    "currency" : "usd",
    "amount" : 1.0,
    "sell" : True
})
print(f"Inserted ID: {result.inserted_id}")

# --- CREATE: insert multiple documents ---
investments.insert_many([
    {"coin_id": "ethereum", "currency": "usd", "amount": 5.0, "sell": False},
    {"coin_id": "solana", "currency": "usd", "amount": 20.0, "sell": False},
])

# --- READ: find all documents ---
print("\nAll investments:")
for doc in investments.find({}): # empty filter = match all
    print(f" {doc['coin_id']:12s} | amount: {doc['amount']} | sell: {doc['sell']}")

# --- READ: find with filter ---
btc = investments.find_one({"coin_id": "bitcoin"})
print(f"\nBitcoin doc: {btc}")

# --- UPDATE: update_many ---
investments.update_many(
    {"sell": False}, # filter: match all docs where sell is False
    {"$set": {"sell": True}} # update: set sell to True
)
print("\nAfter update - sell=True for all:")
for doc in investments.find({}):
    print(f" {doc['coin_id']:12s}: sell={doc['sell']}")

# --- DELETE: delete_one ---
investments.delete_one({"coin_id": "solana"})
```

```
print(f"\nAfter delete — count: {investments.count_documents({})}")
```

Concept

Mongita's API mirrors MongoDB's pymongo driver. `MongitaClientDisk()` stores data as local files (analogous to SQLite). Accessing `client.portfolio` and `db.investments` creates the database and collection automatically if they don't exist — no explicit CREATE statement needed. `insert_one()` accepts a Python dictionary and returns an `InsertOneResult` with the auto-generated `inserted_id`. `find()` returns a cursor (iterator) over matching documents. An empty filter `{}` matches all documents. `update_many()` takes a filter and an update operator — `$set` changes specific fields without replacing the entire document. `delete_one()` removes the first matching document.

Why This Approach?

The document model's key advantage is visible here: documents don't need to follow a fixed schema — each investment could have completely different fields. The `$set` operator is critical: it modifies only the specified fields, leaving all other document content intact. Replacing the whole document would require re-sending all fields. The `find()` cursor is lazy (it doesn't load all documents immediately) — ideal for large collections where loading everything at once would exhaust memory.

Expected Output & Explanation

Inserted ID: 64f... (an ObjectId-like string generated by Mongita)
 All investments: bitcoin (1.0, sell=True), ethereum (5.0, sell=False), solana (20.0, sell=False)
 Bitcoin doc: {'_id': '...', 'coin_id': 'bitcoin', 'currency': 'usd', 'amount': 1.0, 'sell': True}
 After update — all show sell=True (\$set applied to ethereum and solana)
 After delete — count: 2 (solana document removed)

Cell 9 — pymongo — Production MongoDB with Filtering and Operators

```
from pymongo import MongoClient

# Connect to MongoDB server (localhost default port 27017)
client = MongoClient() # or MongoClient('mongodb://localhost:27017/')
db = client.portfolio
investments = db.investments

# Clean slate for demo
investments.drop()
```

```
# Insert sample documents
investments.insert_many([
    {"coin": "bitcoin", "amount": 1.5, "currency": "usd"},
    {"coin": "bitcoin", "amount": 0.5, "currency": "usd"},
    {"coin": "ethereum", "amount": 10.0, "currency": "usd"},
    {"coin": "solana", "amount": 50.0, "currency": "inr"},
])

# --- Filter: $and combining multiple conditions ---
query = {
    "$and": [
        {"coin": "bitcoin"},
        {"amount": {"$gt": 1.0}} # amount greater than 1.0
    ]
}
print("Bitcoin with amount > 1.0:")
for doc in investments.find(query):
    print(f" {doc}")

# --- $inc operator: increment a numeric field ---
investments.update_many(
    {"coin": "ethereum"},
    {"$inc": {"amount": 2.0}} # add 2.0 to existing amount
)
eth = investments.find_one({"coin": "ethereum"})
print(f"\nEthereum after $inc: amount={eth['amount']}") # 12.0

# --- $push: append to an array field ---
investments.update_one(
    {"coin": "bitcoin", "amount": 1.5},
    {"$push": {"tags": "hodl"}}
)

# --- $pull: remove from array field ---
investments.update_one(
    {"coin": "bitcoin", "amount": 1.5},
    {"$pull": {"tags": "hodl"}}
```

```
)
```

Concept

pymongo is the official MongoDB Python driver. It offers the full MongoDB API. `$and` accepts an array of condition objects — all must be satisfied. The `$gt` comparison operator is nested inside the field's query object: `{"amount": {"$gt": 1.0}}`. `$inc` is an atomic increment operator — it adds the specified value to the field's current value without needing to read the current value first, making it safe for concurrent operations. `$push` appends to an array field (creating it if it doesn't exist). `$pull` removes all matching values from an array field. These update operators modify the document in-place rather than replacing it entirely.

Why This Approach?

MongoDB's query operators enable rich, expressive queries without SQL. The `$and` pattern is the MongoDB equivalent of SQL's `WHERE coin='bitcoin' AND amount > 1.0`. The `$inc` operator is essential for counter fields and running totals in high-throughput scenarios — it is **atomic**, meaning multiple concurrent processes can safely increment the same field without race conditions. Array operators (`$push` , `$pull`) are where MongoDB truly shines — managing arrays of embedded data without needing a separate join table as SQL would require.

Expected Output & Explanation

Bitcoin with amount > 1.0:

```
{'_id': ObjectId('...'), 'coin': 'bitcoin', 'amount': 1.5, 'currency': 'usd'}
```

Ethereum after \$inc: amount=12.0 (10.0 + 2.0)

Tags array created by \$push, then emptied by \$pull on the bitcoin document.

Cell 10 — Embedded Documents and Arrays in MongoDB

```
from pymongo import MongoClient
```

```
client = MongoClient()
```

```
db = client.portfolio
```

```
watchlists = db.watchlists
```

```
watchlists.drop()
```

```
# --- Embedded Document ---
```

```
watchlist = {
```

```
    "name": "My Watchlist",
```

```

    "metadata": {
        "currency"    : "USD",
        "description": "Top cryptocurrencies to watch",
        "created_by"  : "Arjun"
    }
}
watchlists.insert_one(watchlist)

# Query using dot notation to access nested field
doc = watchlists.find_one({"metadata.currency": "USD"})
print(f"Found via nested query: {doc['name']}")
print(f"Created by: {doc['metadata']['created_by']}")

# --- Embedded Array of Documents ---
portfolio_doc = {
    "name"    : "My Portfolio",
    "coins"   : [
        {"coin": "bitcoin",   "allocation_pct": 40},
        {"coin": "ethereum",  "allocation_pct": 35},
        {"coin": "solana",    "allocation_pct": 25},
    ]
}
result = watchlists.insert_one(portfolio_doc)

# Add a new coin to the coins array
watchlists.update_one(
    {"name": "My Portfolio"},
    {"$push": {"coins": {"coin": "cardano", "allocation_pct": 0}}}
)

# Query: find portfolio containing bitcoin
found = watchlists.find_one({"coins.coin": "bitcoin"})
print(f"\nCoins in portfolio: {[c['coin'] for c in found['coins']]}")

```

Concept

Embedded documents allow hierarchically related data to be stored within a single MongoDB document. The **metadata** sub-document lives **inside** the watchlist document — no separate table or JOIN needed. MongoDB's **dot notation** (**"metadata.currency"**)

enables querying nested fields directly in the filter object. Embedded arrays of documents — the `coins` array — allow a one-to-many relationship (one portfolio, many coins) to be stored in a single document. The `$push` operator appends to this array. The `"coins.coin"` dot-notation filter matches documents where **any element** of the `coins` array has a `coin` field equal to `"bitcoin"`.

Why This Approach?

Embedded documents eliminate the need for JOINS — arguably the most expensive operation in relational databases. In MongoDB, fetching a watchlist and all its coins is a single document read. In a SQL database, it would require a JOIN across two tables. This **data locality** is MongoDB's core performance advantage for read-heavy workloads. The trade-off: updates to embedded data require finding the parent document first. The choice between embedding and referencing in MongoDB mirrors the denormalization vs. normalization decision in SQL schema design.

Expected Output & Explanation

Found via nested query: My Watchlist (dot-notation filter worked)

Created by: Arjun (nested field accessed with `doc['metadata']['created_by']`)

Coins in portfolio: ['bitcoin', 'ethereum', 'solana', 'cardano'] (cardano added via `$push`)

Part D — SQLAlchemy ORM

Cell 11 — Defining ORM Models with SQLAlchemy 2.0

```
from sqlalchemy import create_engine, String, Float, Integer, ForeignKey
from sqlalchemy.orm import DeclarativeBase, mapped_column, Mapped,
relationship, Session
from typing import Optional, List

# --- Base class: all models inherit from this ---
class Base(DeclarativeBase):
    pass

# --- Portfolio model (parent) ---
class Portfolio(Base):
    __tablename__ = "portfolio"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(64), nullable=False)
```

```
# Relationship: one portfolio -> many investments
investments: Mapped[List["Investment"]] = relationship(
    back_populates="portfolio",
    cascade="all, delete-orphan"
)

def __repr__(self) -> str:
    return f"Portfolio(id={self.id}, name={self.name!r})"

# --- Investment model (child) ---
class Investment(Base):
    __tablename__ = "investment"

    id: Mapped[int] = mapped_column(primary_key=True)
    coin: Mapped[str] = mapped_column(String(32),
    nullable=False)
    currency: Mapped[str] = mapped_column(String(3),
    default="usd")
    amount: Mapped[float] = mapped_column(Float)
    portfolio_id: Mapped[Optional[int]] =
    mapped_column(ForeignKey("portfolio.id"))

    # Back-reference to parent Portfolio
    portfolio: Mapped[Optional["Portfolio"]] = relationship(
        back_populates="investments"
    )

    def __repr__(self) -> str:
        return f"Investment(coin={self.coin!r}, amount={self.amount},
        currency={self.currency!r})"

# --- Create engine (SQLite here; swap URL for PostgreSQL) ---
engine = create_engine("sqlite:///orm_portfolio.db", echo=False)

# Create all tables defined in models
Base.metadata.create_all(engine)
```

```
print("ORM tables created successfully.")
```

Concept

Modern SQLAlchemy 2.0 uses **Mapped type annotations** for columns. `Mapped[int]` declares both the Python type and the SQLAlchemy column type simultaneously. `mapped_column()` replaces the older `Column()` syntax and supports all column constraints. `DeclarativeBase` is the new base class factory — all models inherit from your project-specific `Base`. The `relationship()` call on `Portfolio` declares a one-to-many association — it tells SQLAlchemy that `Portfolio.investments` should contain a list of `Investment` objects. `back_populates` creates the reverse reference: `Investment.portfolio` points back to the parent. `cascade='all, delete-orphan'` means when a `Portfolio` is deleted, all its `Investments` are also deleted automatically. `Base.metadata.create_all(engine)` issues all CREATE TABLE statements.

Why This Approach?

SQLAlchemy 2.0's type-annotated style is fully compatible with mypy and other static type checkers — a significant advantage in large codebases. The `echo=False` parameter on `create_engine` silences SQL logging (set `echo=True` during debugging to see every SQL statement SQLAlchemy generates). The ORM relationship eliminates the need to manually write JOIN queries — `portfolio.investments` gives you a list of `Investment` objects with zero SQL written by the developer. The `ForeignKey("portfolio.id")` in `Investment` is the database-level constraint that enforces referential integrity.

Expected Output & Explanation

ORM tables created successfully.

SQLAlchemy issued these SQL statements internally (visible with `echo=True`):

```
CREATE TABLE portfolio (id INTEGER NOT NULL, name VARCHAR(64) NOT NULL,
PRIMARY KEY (id));
```

```
CREATE TABLE investment (id INTEGER NOT NULL, coin VARCHAR(32) NOT NULL, ...,
portfolio_id INTEGER, FOREIGN KEY(portfolio_id) REFERENCES portfolio(id));
```

Cell 12 — CRUD Operations with SQLAlchemy Session

```
from sqlalchemy import select

# --- CREATE: add objects using session ---
with Session(engine) as session:
    # Create portfolio object
    my_portfolio = Portfolio(name="Crypto Portfolio 2024")
```

```
# Create investment objects (not yet in DB)
bitcoin = Investment(coin="bitcoin", amount=1.5, currency="usd")
ethereum = Investment(coin="ethereum", amount=10.0, currency="usd")
solana = Investment(coin="solana", amount=50.0, currency="usd")

# Attach investments to portfolio via relationship
my_portfolio.investments = [bitcoin, ethereum, solana]

# Add portfolio (cascade adds investments too)
session.add(my_portfolio)
session.commit()
print(f"Created portfolio ID: {my_portfolio.id}")

# --- READ: query with select() ---
with Session(engine) as session:
    stmt = select(Investment).where(Investment.coin == "bitcoin")
    btc_inv = session.execute(stmt).scalar_one()
    print(f"\nFound: {btc_inv}")

# Query all investments, ordered
all_stmt = select(Investment).order_by(Investment.amount.desc())
all_inv = session.execute(all_stmt).scalars().all()
print("\nAll investments (desc by amount):")
for inv in all_inv:
    print(f" {inv}")

# --- UPDATE: modify and commit ---
with Session(engine) as session:
    stmt = select(Investment).where(Investment.coin == "ethereum")
    eth = session.execute(stmt).scalar_one()
    eth.amount = 12.5 # just modify the attribute
    session.commit() # SQLAlchemy detects the change and issues UPDATE
    print(f"\nUpdated ethereum amount: {eth.amount}")

# --- DELETE ---
with Session(engine) as session:
    stmt = select(Investment).where(Investment.coin == "solana")
    sol = session.execute(stmt).scalar_one()
```

```

session.delete(sol)

session.commit()

remaining = session.execute(select(Investment)).scalars().all()

print(f"\nAfter delete — {len(remaining)} investments remain")

```

Concept

The `with Session(engine) as session:` context manager ensures the session is properly closed (and rolled back on error) after each block. `session.add(my_portfolio)` registers the portfolio for insertion — because of the cascade, the three investments are also registered. `session.commit()` issues the INSERT statements and assigns database-generated IDs. The `select()` construct builds a SELECT statement as a Python expression. `scalar_one()` retrieves exactly one result — it raises an exception if 0 or 2+ rows match (fail fast on ambiguity). `.scalars().all()` unwraps the result set into a plain list of model instances. For UPDATE, SQLAlchemy uses its **identity map**: it tracks every loaded object, detects attribute changes, and automatically generates the minimal UPDATE SQL on commit. No explicit UPDATE statement is needed.

Why This Approach?

SQLAlchemy's session-based CRUD has several advantages over raw SQL. Updates require no SQL string — modify Python attributes and commit. The ORM tracks changes automatically (**change tracking**). Relationships allow navigating from a Portfolio to its Investments using `portfolio.investments` — no JOIN SQL required. The `with Session` context manager pattern prevents connection leaks. The `cascade='all, delete-orphan'` on the relationship means deleting a Portfolio automatically deletes all its Investment children — no manual cleanup required.

Expected Output & Explanation

Created portfolio ID: 1 (auto-assigned by SQLite)
 Found: Investment(coin='bitcoin', amount=1.5, currency='usd')
 All investments (desc by amount):
 Investment(coin='solana', amount=50.0, currency='usd')
 Investment(coin='ethereum', amount=10.0, currency='usd')
 Investment(coin='bitcoin', amount=1.5, currency='usd')
 Updated ethereum amount: 12.5 (SQLAlchemy generated UPDATE automatically)
 After delete — 2 investments remain (solana removed)

Quick Reference — Database Operations in Python

Task	Library & Pattern
------	-------------------

Connect to SQLite	<code>sqlite3.connect("file.db") + .cursor()</code>
Execute SQL	<code>cursor.execute("SQL", (param,))</code>
Parameterized insert	<code>cursor.execute("INSERT ... VALUES (?,...)", (val,))</code>
Fetch all rows	<code>cursor.fetchall()</code> — list of tuples
Fetch one row	<code>cursor.fetchone()</code> — single tuple
Named column access	<code>database.row_factory = sqlite3.Row</code>
Dataclass row factory	Custom factory: <code>Investment(**dict(zip(cols, row)))</code>
Connect to PostgreSQL	<code>psycopg2.connect(host=, database=, user=, password=)</code>
Mongita CRUD	<code>insert_one(), find(), update_many(), delete_one()</code>
pymongo filter	<code>{"\$and": [{"field": val}, {"field2": {"\$gt": n}}]}</code>
MongoDB update	<code>{"\$set": {}}, {"\$inc": {}}, {"\$push": {}}, {"\$pull": {}}</code>
Embedded doc query	<code>find({"nested.field": value})</code> — dot notation
SQLAlchemy model	<code>class Model(Base): __tablename__ = ...; col = mapped_column(...)</code>
Create tables (ORM)	<code>Base.metadata.create_all(engine)</code>
ORM session CRUD	<code>with Session(engine) as s: s.add(obj); s.commit()</code>
ORM query	<code>session.execute(select(Model).where(...)).scalar_one()</code>
ORM relationship	<code>relationship(back_populates=...) + ForeignKey(...)</code>

Chapter Summary

This chapter provided a comprehensive tour of Python's database ecosystem — from lightweight embedded storage to enterprise-grade relational servers and flexible document databases:

1. **SQLite with `sqlite3`** — The built-in, zero-configuration relational database. Full CRUD with parameterized queries, the connection-cursor model, row factories, and dataclass integration.
2. **SQL Injection Prevention** — Parameterized queries with `?` placeholders are the only safe way to incorporate user input into SQL. This rule is absolute and non-negotiable.
3. **Row Factories and Dataclasses** — Transforming raw tuple rows into named, typed Python objects eliminates fragile positional indexing and bridges the database and application layers cleanly.
4. **PostgreSQL with `psycopg2`** — A network-attached enterprise relational server. The DB-API 2.0 interface mirrors `sqlite3`, making migration straightforward. PostgreSQL uses `%s` as the parameterized query placeholder.

5. **NoSQL with Mongita and pymongo** — Document-oriented storage with schema flexibility. Full CRUD, embedded documents, arrays, and MongoDB's \$ operator-based query and update language.
6. **SQLAlchemy ORM** — Python's most powerful database toolkit. Model-based table definition, session management, automatic change tracking, and relationship() for navigation-free joins. Database-portable by design.

What's Next?

In the next chapter, we will explore data manipulation with pandas — building on the database skills developed here by reading SQL query results directly into DataFrames, performing advanced data transformation pipelines, and writing processed data back to databases using pandas' built-in SQL integration.